GOLD: Introduction to Java (Wed Aug 26)

Types

Every value and variable has a type

What is a **type**?

A **type** is two things:

1. The **set of values** you're allowed to have
2. The **set of operations** you're allowed to do with them

For now, we'll focus on just these types:

Type	What it holds	Example literals
`int`	whole numbers	`2`, `-7`, `0`
`double`	numbers with a decimal point	`8.50`, `-0.5`, `3.14`
`String`	text in double quotes	`"Burger"`, `"hello"`

A **literal** is a human-readable representation stored in source code.

Our burger stand will need all three of these types:

```
void main() {
    String item = "Burger";
    int burgers = 2;
    double price = 8.50;

    // Some operations are not allowed on some types:
    IO.println( item * 2 );
}
```

Declaration

Declaring variables

Java requires you to **declare** a variable before you use it.

To **declare** a variable means name it and give it a type.

```
int burgers;          // declare only (box exists, nothing stored yet)
int burgers = 2;      // declare and initialize at the same time (preferred)
```

Naming rules and conventions:

- Names start with a letter; then letters, digits, `_`
- Use `lowerCamelCase`: `burgers`, `friesPrice`, `totalCost`
- Use names that say what the box is *for:* `burgers`, not `b`

What happens if we don't initialize or set a variable before trying to use its value?

```
void main() {
    int burgers;
    IO.println(burgers);
}
```

Java refuses to run it. The mistake is caught **before** the program starts:

```
B.java:3: error: variable burgers might not have been initialized
    IO.println(burgers);
               ^
```

The compiler is protecting us: an empty box has nothing meaningful to print.

Arithmetic

Arithmetic

To create a receipt we'll need to do arithmetic on prices

- How much does 2 burgers cost?
- What is the total price of the order?

```
void main() {
    int burgers = 2;
    double price = 8.50;
    double cost = burgers * price;
    IO.println(cost);
}
```

We have the usual mathematical operations: `+` `-` `*` `/` and one unfamiliar one: `%` (remainder)

Same order of operations as math: `*`, `/`, (and `%`) happen before `+`, `-`. Parentheses override, just like math:

```
2 + 3 * 4       // is 14
(2 + 3) * 4     // is 20
```

Integer Division

First Surprise: Integer Division

When you divide an `int` by an `int`, Java gives back an `int`. The fractional part is **thrown away**.

```
10 / 5     // is 2
13 / 4     // is 3   (not 3.25!)
1 / 2      // is 0   (!!)
```

We'll see how to work around this next lecture.

This is a language choice. Other programming languages (such as Python) choose to return a decimal.

If we have at least one `double` involved then we get an accurate double:

```
void main() {
    double totalBurgerCost = 35.00;
    int burgers = 8;
    IO.println(totalBurgerCost / burgers); // 4.375 (double / int)
    IO.println(35 / 8);                    // 4     (int / int throws away .375)
}
```

What was that `%` operation?

`%` gives you the part that was thrown away during division: the remainder:

```
17 % 3     // 2     (17 = 3 * 5, remainder 2)
13 % 4     // 1
```

We've used this operation without realizing it: `% 12` is clock arithmetic.

Your Program Is a File

Everything you just ran in the browser lives in an ordinary **file**.

- The file is named after the program: `Hello.java`
- For now we can name it whatever we like, as long as it ends in `.java`
- Inside the file is exactly the text you saw.

On a lab computer, you run it with **one command** in the terminal:

```
$ java Hello.java
Hello, World!
```

That's the whole loop:

edit the file → run the file → read the output.

Every programming tool (the terminal, IntelliJ soon, Ed) is a different view of this loop (perhaps with an extra step).

In your first homework you'll create and run your first file yourself.

A Program with Two Bugs

This week we'll pretend we're writing software for a part-time job at Boiler Burgers.

We need a program that can produce a receipt.

```
void main() {
    double burgerPrice = 8.50;
    double friesPrice = 3.29;
    int burgers = 2;
    int fries = 3;
    double total = burgers * burgerPrice + fries * friesPrice;
    IO.println("== Boiler Burgers ==");
    IO.println(burgers + " x Burger: $" + burgers * burgerPrice);
    IO.println(fries + " x Fries:  $" + fries * friesPrice);
    IO.println("Items: " + burgers + fries);
    IO.println("Total: $" + total);
}
```

Is this what we expect?

Here's the output:

```
== Boiler Burgers ==
2 x Burger: $17.0
3 x Fries:  $9.870000000000001
Items: 23
Total: $26.87
```

Two bugs:

- Three fries cost **$9.87**, not $9.870000000000001.
- And 2 burgers plus 3 fries is **5 items**, not 23.

The code looks completely reasonable, but AI output often looks reasonable before something goes wrong.

Don't Worry if you Don't Understand the Code Yet

- If you've never written a line of code before, you are in the right room.
- We'll go through every bit of the program above

- Next lecture you'll learn exactly what's wrong with this code.

The Brief Outline of this Course

- By the end of the week, we will have written Java that does something real.
- By the end of the course, **you will be able to look at code and know how to determine whether it's correct.**
- Understanding how and why the code works is the "real" skill.
- Even if you write code with AI in the future, the accountability still belongs to you.

Arithmetic

Question *Submitted Aug 26th 2026 at 3:51:37 pm*

Trap of the Day: what does this print?

```
IO.println(1 / 2 * 2.0);
```

- 1.0
- 0.0
- 0
- 2.0

Building Strings

Joining text with `+`

When `+` has a String on either side, it **joins** (technical term: concatenates) them into one String.

Numbers get converted to text automatically.

```
void main() {
    int burgers = 2;
    double price = 8.50;
    IO.println(burgers + " x Burger at $" + price);
}
```

Output:

```
2 x Burger at $8.5
```

Notes on the code:

- The output was `$8.5` not `$8.50`. The **value**, which is the data stored in the computer, is different than the literal we write.
    - The trailing zero is not remembered.

- The operation `+` is doing two different jobs! How does Java know which one to do?

Correcting the Receipt

Let's fix the receipt

Earlier we looked at this incorrect program:

```
void main() {
    double burgerPrice = 8.50;
    double friesPrice = 3.29;
    int burgers = 2;
    int fries = 3;
    double total = burgers * burgerPrice + fries * friesPrice;
    IO.println("== Boiler Burgers ==");
    IO.println(burgers + " x Burger: $" + burgers * burgerPrice);
    IO.println(fries + " x Fries:  $" + fries * friesPrice);
    IO.println("Items: " + burgers + fries);
    IO.println("Total: $" + total);
}
```

Line 10 is the problem here:

`IO.println("Items: " + burgers + fries);`

should give us `Items: 5` not `Items: 23`

What went wrong?

- Like in high school math, `+` is performed left-to-right
- We can use parentheses to control the order of evaluation

▶ Expand

Here's the corrected receipt — the Items line is fixed, but look at the fries line:

```
void main() {
    double burgerPrice = 8.50;
    double friesPrice = 3.29;
    int burgers = 2;
    int fries = 3;
    double total = burgers * burgerPrice + fries * friesPrice;
    IO.println("== Boiler Burgers ==");
    IO.println(burgers + " x Burger: $" + burgers * burgerPrice);
    IO.println(fries + " x Fries:  $" + fries * friesPrice);
    IO.println("Items: " + (burgers + fries));
    IO.println("Total: $" + total);
}
```

We haven't talked about the `$9.870000000000001` error yet.

Why do we run into this tiny rounding error here?

Concept: Representation

CS Concept: Representing Real Numbers

The broken fries line: `$9.870000000000001` *can't* be traced line by line.

- A real number type like `double` can't store most money amounts exactly.
- It represents a *subset* of the real numbers
- There is limited space (0s and 1s) in computer memory and too many real numbers,
- ... so we end up with the closest number we can represent

We'll talk about why later, but for now we need to remember that most money values can't be represented exactly.

What Does the Program Print?

Question *Submitted Aug 26th 2026 at 3:57:50 pm*

What does the following program output?

```
void main() {
    int x = 4;
    int y = x + 2;
    x = x * y;
    IO.println(x + y);
    IO.println("x" + y);
}
```

- ○
```
30
30
```

- ○
```
10
x6
```

- ◉
```
30
x6
```

- ○
```
30
x24
```

- ○
```
246
x6
```

User Input

Talking to the Program

This is where we are now:

```
void main() {
    String item1 = "Burger";
    int qty1 = 2;
    double price1 = 8.50;

    String item2 = "Fries";
    int qty2 = 3;
    double price2 = 3.29;

    IO.println("== Boiler Burgers ==");
    IO.println(qty1 + " x " + item1);
    IO.println(qty2 + " x " + item2);
    IO.println("Items: " + (qty1 + qty2));
    IO.println("Total: $" + (qty1 * price1 + qty2 * price2));
}
```

The program is logically correct, and the output happens to display cleanly.

The representation gremlin only bites some values.

What's missing?

A real receipt needs to take **input:** data from the user or data about the order to produce different receipts.

Basic Input

We've been using `IO.println()` to send **output** to the screen.

The companion for input is `IO.readln()`

```
void main() {
    String name = IO.readln("Name for the order? ");
    IO.println("Welcome to Boiler Burgers, " + name + "!");
}
```

What happens when we run it?

▶ Expand

Note that `IO.readln("Name...")` produces a value.

Reading Numbers

Reading numbers takes one extra step

What if we want to read in a number too? Let's try it:

```
void main() {
    String order = IO.readln("How many burgers? ");
    double orderTotal = order * 8.50;
    IO.println(orderTotal + " cost of burgers ordered");
}
```

This program doesn't even compile: `order` is a `String`, and `String * double` is not an allowed operation.

Correct code:

▶ Expand

The method `IO.readln` **always** gives you back a `String`, even if the user typed `4`.

The String `"4"` and the int `4` are different values with different types.

You'll learn later about how the representation inside the computer is very different.

We need to use a special method to convert these to integers:

- `Integer.parseInt(text)` converts a String like `"4"` into the int `4`
- `Double.parseDouble(text)` converts a String like `"8.50"` into the double `8.5`

Exceptions and Crashes

What if the user types nonsense?

What if the user types nonsense into the program? Or doesn't understand the correct formatting?

```
void main() {
    String text = IO.readln("How many burgers? ");
    int burgers = Integer.parseInt(text);
    IO.println("That's " + (burgers * 8.50) + " dollars.");
}
```

▶ Expand

The program crashes: Java refuses to guess what `"four"` means as an `int`.

For now: that's fine.

Handling bad input gracefully is a skill we'll learn later in the course.

Mixing Types

Mixing `int` and `double`

We've been doing this since the beginning:

```
int burgers = 4;
double price = 8.50;
double cost = burgers * price;   // int * double → double
```

The rule: when an `int` meets a `double`, Java **promotes** the `int` to a `double` first. Nothing is lost going that direction.

```
3 + 5.0     // becomes 3.0 + 5.0 → 8.0
1 / 2.0     // becomes 1.0 / 2.0 → 0.5
1 / 2       // both ints → 0   (the division trap!)
```

We can also store an `int` literal in a `double` variable:

```
void main() {
    double price = 3;        // the int 3 is promoted to 3.0
    IO.println(price);       // prints 3.0
    // int burgers = 3.99;   // ERROR: a double into an int would lose information
}
```

Making Decisions

Making decisions: `if` / `else`

Boiler Burgers runs a promotion: **orders of $25 or more get $5 off.**

```
if (total >= 25.0) {
    total = total - 5.0;
    IO.println("Big order! $5 off.");
} else {
    IO.println("Spend $25 to get $5 off next time!");
}
```

The condition in parentheses is either **true** or **false**. We'll talk a lot more soon about what we can use for our condition, but for now we're going to compare two numbers.

- True → we run the first block.
- False → we run the `else` block.

- Either way, the program continues after the `}`.

Comparison Operators

Comparison operators

Operator	Means
`==`	equal to
`!=`	not equal to
`<`	less than
`<=`	less than or equal
`>`	greater than
`>=`	greater than or equal

Be careful: `=` stores, `==` compares.

⚠ **One equals to store, two equals to compare.**

The condition is a value too: `boolean`

A fourth type, `boolean`, has exactly **two values**: `true` and `false`.

```
double total = 26.87;
boolean bigOrder = total >= 25.0;
IO.println(bigOrder);              // prints: true
```

`total >= 25.0` isn't magic syntax inside an `if` .

This is an **expression** that produces a value (like `2 + 3` produces `5`).

The `if` just looks at that value.

The Interactive Register

Let's build the register together now:

```
void main() {
    final double BURGER_PRICE = 8.50;
    final double FRIES_PRICE = 3.29;
    final double DISCOUNT_THRESHOLD = 25.0;
    final double DISCOUNT_AMOUNT = 5.0;

    String name = IO.readln("Name for the order? ");
    int burgers = Integer.parseInt(IO.readln("How many burgers? "));
    int fries = Integer.parseInt(IO.readln("How many fries? "));

    double total = burgers * BURGER_PRICE + fries * FRIES_PRICE;

    IO.println("== Boiler Burgers ==");
    IO.println("Order for " + name);
    IO.println(burgers + " x Burger: $" + burgers * BURGER_PRICE);
    IO.println(fries + " x Fries:  $" + fries * FRIES_PRICE);

    if (total >= DISCOUNT_THRESHOLD) {
        total = total - DISCOUNT_AMOUNT;
        IO.println("Big order! $" + DISCOUNT_AMOUNT + " off.");
    } else {
        IO.println("Spend $" + DISCOUNT_THRESHOLD + " to get $" + DISCOUNT_AMOUNT + "
    }
    IO.println("Total: $" + total);
}
```

One more lesson:

Don't stick magic numbers like 25.0 or 5.0 in our code.

No one will remember what they're for, and they're hard to track down later.

Instead make a **constant** in a variable declaration using the keyword `final`.

`final double DISCOUNT_THRESHOLD = 25.0;`
`final double DISCOUNT_AMOUNT = 5.0;`

`final` means the box is **sealed**: store once, never change.

A `final` variable is called a **constant**, and by convention its name is `ALL_CAPS_WITH_UNDERSCORES`.